

BlueVantage — Project Document

Version 1.0 | **Compiled:** April 25, 2026 | **Location:** CometLab, Noordwijk, NL

TABLE OF CONTENTS

1. Project Overview & Vision
2. Problem Statement (Lean Startup Narrative)
3. Market Landscape & Competitive Analysis
4. Blue Ocean Strategy
5. Business Model & Unit Economics
6. Product Specification & Feature Map
7. Geospatial Data Pipeline
8. ML & Prediction Engine — Full Technical Design
9. Route Optimization Logic
10. Hackathon MVP — 6–8 Hour Build Plan
11. Team Structure & Task Delegation
12. Open Questions & Risks
13. Honest Data Limitations & Framing

1. PROJECT OVERVIEW & VISION

Product Name: BlueVantage

Tagline: *Informed fishing, fairly allocated, sustainably capped.*

Stage: Pre-MVP / Hackathon Build

Market: Netherlands — North Sea commercial fishery

Hackathon Category: Challenge #2 — Tracking and Prediction

BlueVantage is a **fishing zone intelligence marketplace** — combining environmental satellite data, predictive species habitat modeling, and a self-allocation bidding system to help Dutch commercial fishers maximize catch yield, minimize fuel waste, stay MPA-compliant, and sustainably share the sea without crowding each other out.

From: Guesswork-driven fishing trips based on generational intuition

To: Data-informed, competitively allocated, sustainability-compliant zone selection

2. PROBLEM STATEMENT (LEAN STARTUP NARRATIVE)

The Problem

Dutch commercial fishers operating in the North Sea lose an estimated **€180M annually** to inefficient fishing trips. The root cause is information asymmetry: fishers depart port without reliable, real-time knowledge of where their target species are concentrated, what sea conditions are like at specific zones, or which zones are already crowded by competing vessels.

Three compounding pain points:

- **Yield uncertainty:** 30–40% of trips underperform expectations. Fishers rely on experience and word-of-mouth, not data.
- **Fuel waste:** A beam trawler burns ~€800–1,200 in fuel per trip. Searching for fish rather than going where they are destroys margins.
- **Compliance risk:** Marine Protected Areas (MPAs — Natura 2000, Dogger Bank, Klaverbank) carry fines of up to €50,000 for violations. Many fishers lack real-time boundary awareness.

The Hidden Problem: Crowding

If a platform simply broadcasts the same "best zone" to all fishers in a region, it:

- Destroys the yield advantage it promised
- Causes local overfishing within days
- Erodes fisher trust irreversibly

This is why recommendation alone is insufficient. **The solution must include allocation.**

The Solution Hypothesis

A marketplace where fishers access data-scored fishing zones and **self-allocate** via a bidding system — with hard zone capacity caps — preserves the yield advantage while preventing crowding and distributing access fairly.

Customer Segments

Segment	Size	Pain Level
Owner-operator (1–2 vessels)	~400 in NL	Very high
Small cooperative (3–10 vessels)	~80 cooperatives	High
Fleet manager (10+ vessels)	~30 companies	High (operational complexity)

Beachhead: Solo owner-operators out of Scheveningen, IJmuiden, Den Helder — smartphone-native, cost-sensitive, trip-by-trip decision makers.

Unfair Advantage

- First mover building a **proprietary logbook dataset** via fisher incentive program
- Academic proximity to NIOZ, TU Delft oceanography, ICES data infrastructure
- The data flywheel: more fishers → more logbook data → better predictions → more fishers

3. MARKET LANDSCAPE & COMPETITIVE ANALYSIS

Market Size (Netherlands)

Layer	Figure
NL licensed commercial fishing vessels	~800 active
Average vessel revenue	€180,000–€450,000/yr

Total NL fishing industry revenue	~€450M/yr
Addressable vessels (>10m, North Sea)	~400
TAM (SaaS at €200/mo per vessel)	~€960k ARR at full penetration
Realistic 3-yr target (150 vessels)	~€270k ARR

Competitive Landscape

Competitor	What they do	Gap
Satlink	Vessel monitoring hardware + basic tracking	No zone intelligence, no species prediction
Pelagic Data Systems	Sensor hardware for small-scale fishers	Hardware-only, no software intelligence layer
OceanMind	MPA compliance monitoring for regulators	B2G, not fisher-facing
Fiskeridir (Norway)	Norwegian quota management platform	NL market untouched, no zone prediction
MarineTraffic	AIS vessel tracking	No fishing intelligence, no allocation
Traditional VMS providers	Mandatory tracking compliance	Pure compliance, zero intelligence layer

Gap: No player combines (1) zone-level species habitat prediction + (2) self-allocation bidding + (3) MPA compliance + (4) logbook incentive flywheel in a single fisher-facing SaaS tool for the Dutch market.

4. BLUE OCEAN STRATEGY

Strategy Canvas — Key Competing Factors

Factor	Traditional VMS	MarineTraffic	BlueVantage
Compliance tracking	High	Medium	High
Zone intelligence	None	None	Very High

Species prediction	None	None	High
Allocation / anti-crowding	None	None	Core feature
MPA awareness	Low	Medium	Very High
Fisher revenue optimization	None	None	Core value prop
Data reciprocity / flywheel	None	None	Core mechanic
Price	Low (mandated)	Low	Medium

ERRC Grid

Eliminate	Reduce	Raise	Create
Passive broadcasting of zone tips	Price vs. hardware solutions	MPA compliance certainty	Zone self-allocation marketplace
Generic "fish finder" features	Setup complexity	Yield predictability	Logbook data flywheel
		Species-level granularity	Trip economics dashboard

5. BUSINESS MODEL & UNIT ECONOMICS

Subscription Tiers

Tier	Price	Zone Access	Species	Bidding	Notifications
Free	€0	Read-only map	1 species filter	None	Weekly email
Standard	€150/mo	5 slots/week	5 species	First-come slots	Weekly push
Pro	€250/mo	15 slots/week	5 species + custom	Credit bidding	Real-time + 48h alerts
Fleet	€600/mo	Unlimited multi-vessel	All species	Priority credits	Fleet briefing dashboard

Unit Economics

Metric	Value	Notes
ARPU (blended)	€200/mo	Mix of Standard + Pro
COCA (Cost of Customer Acquisition)	~€800	Field sales + cooperative partnerships
Gross Margin	~78%	API costs + hosting ~€44/mo per customer
LTV (3yr, 25% churn)	~€7,200	Conservative vs. earlier €9,600 (20% churn)
LTV:CAC Ratio	~9x	Healthy for B2B SaaS
Payback Period	~4 months	
Breakeven (vessels)	~35 vessels at Standard	Covers €63k/yr fixed costs

COGS Breakdown (per vessel per month)

Item	Cost
CMEMS API (Copernicus)	Free (EU funded)
Open-Meteo Marine API	Free tier
AIS data (MarineTraffic API)	~€8–15
Hosting (Vercel + Railway + Postgres)	~€12
ICES data	Free
ML compute (inference)	~€5
Support + ops allocation	~€10
Total COGS per vessel/mo	~€35–42

Revenue Projection

Year	Vessels	Blended ARPU	ARR
Y1	30	€180	€64,800
Y2	80	€200	€192,000
Y3	150	€210	€378,000

Churn Stress Test

- At 15% annual churn: LTV = €9,600, LTV:CAC = 12x
- At 25% annual churn: LTV = €7,200, LTV:CAC = 9x
- At 35% annual churn: LTV = €5,100, LTV:CAC = 6.4x — still viable but growth slower

6. PRODUCT SPECIFICATION & FEATURE MAP

Core Product Principles

Principle	Implication
No crowding by design	Zones have hard capacity caps — recommendations never broadcast universally
Fisher autonomy	Bidding is self-directed; platform informs, not dictates
Data reciprocity	Fishers who share logbook data get more from the platform
Compliance built-in	MPA boundaries are non-negotiable hard blocks
Progressive complexity	MVP is simple map + zone cards; complexity layers in post-launch

User Personas

Primary: "Jan" — Vessel Owner-Operator

- 1–2 vessels, North Sea out of IJmuiden / Scheveningen / Den Helder
- Targets 4–5 species seasonally
- Burns €800–1,200/trip on fuel; 30–40% of trips underperform
- Smartphone-native, uses WhatsApp for fleet comms
- Wants: confident zone choice before leaving port, fewer empty nets

Secondary: "Roos" — Fleet Manager

- Manages 5–15 vessels for cooperative/family business

- Needs: aggregate view, multi-vessel bid management, compliance status
- Wants: dashboard, exportable reports

Tertiary: Regulator / Sustainability Partner (future)

- RWS, NVWA, fishing cooperatives
- Wants: MPA compliance data, stock pressure heatmaps

FEATURE MAP BY RELEASE PHASE

● MVP — Hackathon Demo

M1 — Zone Intelligence Map

- Interactive North Sea map (Mapbox GL JS / [Deck.gl](#))
- H3 hexagonal grid overlay, resolution 5 (~25km² cells)
- Color-coded Zone Score (0–100): green → yellow → red
- MPA zones: hard red-hatched exclusion areas (non-clickable)
- AIS vessel density: static layer (15-min refresh)

M2 — Zone Card

On hex click, side panel shows:

- Zone Score (0–100 with breakdown)
- Predicted dominant species (e.g. 62% Plaice, 28% Sole, 10% Other)
- Estimated biomass window (e.g. 400–700 kg/vessel-day)
- Current allocation: X of Y slots filled (progress bar)
- MPA status: Clear / Warning / Blocked
- Weather: wave height + wind speed for next 48h
- Distance from home port
- "Bid for this zone" button (disabled if full or MPA)

M3 — Species Preference Onboarding

- Step 1: Home port (Scheveningen / IJmuiden / Den Helder)
- Step 2: Target species (multi-select: Plaice, Sole, Herring, Cod, Mackerel)
- Step 3: Vessel/gear type (beam trawl / otter trawl / gillnet)
- Profile persists; filters zone scoring to fisher context

M4 — Self-Allocation Bidding

- Each zone has capacity cap (max concurrent vessels by estimated stock pressure)
- Fisher selects zone + time window (Morning / Full Day / Overnight)
- MVP: first-come-first-served slot reservation (no financial bid yet)
- Confirmation: slot number, zone summary, departure window
- Slot count updates in real time
- Cancellation allowed up to X hours before window

M5 — MPA Alert System

- All recommendations exclude MPAs by default (hard filter in scoring engine)
- Zone card for MPA hex: "⊖ Marine Protected Area — fines up to €50,000"
- Static MPA polygons from EMODnet / RWS Natura 2000

M6 — Logbook Data Incentive (Lightweight)

- Post-trip: simple form (zone, species, weight, gear)
- On submit: earns +5 priority bid credits
- Counter visible in header
- Demonstrates data flywheel concept

🌀 V1.1 — Post-Hackathon (Month 1–3)

V1 — Live AIS-Driven Zone Downgrading

- Real-time AIS ingestion (MarineTraffic / AISHub API)
- Zone score dynamically penalized as vessels approach capacity
- Crowding pressure visible to all browsing fishers

V2 — Credit-Based Financial Bidding

- Zone credits earned via logbook sharing + subscription tier
- Blind auction for contested zones (24h before window)
- Ethical guardrail: non-transferable credits, no secondary market

V3 — Quota Integration

- Fisher inputs annual ITQ quota by species
- Platform tracks quota consumption via logbook submissions
- Zone recommendations weighted by remaining quota
- "At current pace, sole quota runs out in 6 weeks"

V4 — Tiered Push Notifications

- Standard: weekly zone forecast
- Pro: 48h push when premium zones open, real-time cancellation alerts
- Fleet: aggregate fleet briefing per vessel per week

● V2.0 — Scale Phase (Month 4–12)

S1 — Fleet Dashboard: Multi-vessel management, zone assignment, aggregate quota, exportable reports

S2 — Predictive Model Upgrade: Spatial ML (Gaussian Process / GNN over H3 graph), 10+ species, quarterly retraining

S3 — Sustainability Reporting: CO₂ per trip, MPA events avoided, sustainability certificate, B2G data sales to RWS/EU

S4 — Market Price Integration: Fish auction price feeds (Urker Visafslag, IJmuiden), estimated trip revenue per zone card, price spike alerts

Feature Priority Matrix

Feature	Impact	Effort	Phase
H3 Zone Map + Scoring	★★★★★	High	MVP
Zone Card (species, yield, slots)	★★★★★	Medium	MVP
MPA Hard Exclusion Layer	★★★★☆	Low	MVP
Species Preference Onboarding	★★★★☆	Low	MVP
Slot-based Bidding	★★★★★	Medium	MVP
Logbook Incentive (manual)	★★★★☆	Low	MVP
Live AIS Zone Downgrading	★★★★★	High	V1.1
Credit-based Financial Bidding	★★★★☆	High	V1.1
Quota Integration	★★★★☆	Medium	V1.1
Fleet Dashboard	★★★★☆	High	V2.0
Spatial ML Model	★★★★★	Very High	V2.0
Sustainability Reporting	★★★☆☆	Medium	V2.0
Market Price Integration	★★★★☆	Medium	V2.0

7. GEOSPATIAL DATA PIPELINE

Data Sources

Source	Data Type	Cadence	Access
CMEMS (Copernicus Marine)	SST, Chlorophyll-a, Salinity, Currents	Daily rasters	Free EU API
KNMI / Open-Meteo Marine	Wind, Wave height, Visibility	48h forecast	Free
AISHub / MarineTraffic API	Vessel positions, speed, heading	15-min refresh	Free tier / paid
ICES DATRAS	Historical trawl survey catch records	Annual	Free, datras.ices.dk

Source	Data Type	Cadence	Access
STECF / Eurostat	Aggregated landings by ICES rectangle	Annual	Free
EMODnet / RWS	MPA polygons, Bathymetry, Substrate type	Static	Free GIS layers
Fisher Logbooks	Catch location, species, weight, gear	Per trip	Via incentive program

Pipeline Architecture

[STAGE 1 — INGESTION & HARMONIZATION]

Person 3 (Pipeline A):

- Pull CMEMS SST + chlorophyll via copernicusmarine Python client
 - NL North Sea bbox: lat 51.5–56.0, lon 2.0–7.5
 - Monthly averages 2022–2024 → netCDF → pandas DataFrame
- Pull Open-Meteo Marine API → wave height, wind per grid point
 - weather_grid.csv
- Pull EMODnet WFS → MPA polygons
 - mpa_polygons.geojson

Person 4 (Pipeline B):

- Generate H3 hex grid over NL North Sea
 - resolution 5 (~25km² per cell) → ~80–120 hex cells
 - h3.polyfill() on bbox polygon
- Spatial join all data sources to hex centroids
 - For each hex: nearest SST, chlorophyll, depth, wave, wind
 - MPA check: does centroid fall inside any MPA polygon?
 - ICES CPUE: average historical catch rate for hex region
- Output: hex_features.csv (one row per hex, all features)

↓

[STAGE 2 — PREDICTION ENGINE]

Person 2 (ML):

- Input: hex_features.csv
- Train XGBoost model on ICES DATRAS labeled records
- Run inference over all hex cells
- Apply zone score formula
- Output: zones.json

↓

[STAGE 3 — SERVING LAYER]

- FastAPI endpoint: GET /zones
- Returns ranked hex list with scores, species probs, slot availability
- Deployed on Railway.app (free, 2 min deploy)
- Or: simply commit zones.json to repo (static fallback)

↓

[STAGE 4 — FRONTEND VISUALIZATION]

Person 1 (Web):

- Mapbox GL JS + deck.gl H3HexagonLayer
- Color ramp: green (80–100) → yellow (50–79) → red (<50)
- MPA overlay: red hatched polygon layer (always on top)
- AIS layer: vessel dots with MMSI tooltip
- Zone Card: triggered on hex click
- Bidding flow: modal → confirm → slot reserved

8. ML & PREDICTION ENGINE — FULL TECHNICAL DESIGN

The Core Scientific Basis

Fish don't distribute randomly. Each commercially targeted species has a known **habitat fingerprint** — a set of environmental conditions it gravitates toward. These are well-studied in fisheries science:

Species	Preferred SST	Preferred Depth	Substrate	Chlorophyll Signal	Peak Season
Plaice	6–12°C	20–50m	Sandy/muddy	Medium	Spring–Autumn
Sole	10–18°C	15–40m	Sandy	Low–Medium	Summer
Cod	2–10°C	50–200m	Mixed	High	Winter–Spring
Herring	6–14°C	20–100m	Open water (pelagic)	Very high	Autumn spawning
Mackerel	10–18°C	Surface–100m	Pelagic	High	Summer

Why Only 5 Species (Out of ~200 in North Sea)?

The North Sea contains ~200+ species but commercially:

Top 5 species = ~85% of Dutch commercial catch by weight

Plaice (schol) ~35%
Herring (haring) ~22%
Sole (tong) ~14%
Mackerel (makreel) ~9%
North Sea shrimp ~5%

All other species ~15%

Modeling 5 species + an "Other" class covers the full commercial revenue picture. This is consistent with how ICES itself manages stocks — focusing on ~30 assessed stocks, not 200.

Feature Vector (Input to Model)

features = {
 'sst_celsius': float, # sea surface temperature (CMEMS)
 'chlorophyll_mg_m3': float, # phytoplankton density (CMEMS)
 'depth_m': float, # bathymetry (EMODnet)
 'substrate_type': int, # encoded: sandy=0, gravel=1, muddy=2 (EMODnet)
 'wave_height_m': float, # (Open-Meteo Marine)

```

'wind_kmh':      float, # (Open-Meteo Marine / KNMI)
'season_week':   int,   # 1-52
'distance_port_km': float, # from nearest NL home port
'historical_cpue': float, # catch per unit effort index (ICES DATRAS)
}

```

Model Output (Per Hex Cell)

```

{
  'p_plaice': 0.62, # probability of plaice dominance
  'p_sole': 0.28,
  'p_cod': 0.08,
  'p_herring': 0.02,
  'p_mackerel': 0.00,
  'yield_kg_mean': 550, # expected catch kg per vessel-day
  'yield_kg_std': 130, # uncertainty range
}

```

Zone Score Formula (Deterministic Layer on Top of ML)

```

def compute_zone_score(zone, fisher):
    # 1. Species match: does predicted top species match fisher preference?
    species_match = zone.p_target_species[fisher.target_species]

    # 2. Yield score: normalized 0-1 against max possible in dataset
    yield_score = zone.yield_kg_mean / MAX_YIELD_IN_DATASET

    # 3. Weather score: inverse penalty for bad conditions
    weather_score = max(0, 1 - (zone.wave_height_m / 4.0)
                        - (zone.wind_kmh / 80.0))

    # 4. Historical CPUE score
    cpue_score = zone.historical_cpue # already normalized 0-1

    # 5. Distance penalty: longer trip = lower score
    distance_penalty = zone.distance_port_km / MAX_DISTANCE_KM

```

```

score = (
    0.35 * species_match +
    0.25 * yield_score +
    0.20 * weather_score +
    0.15 * cpue_score -
    0.05 * distance_penalty
) * 100

# Hard override: MPA zones always score 0
if zone.is_mpa:
    return 0

return round(max(0, min(100, score)), 1)

```

What ML Model to Use

For hackathon: XGBoost (MultiOutputRegressor)

```

from xgboost import XGBRegressor
from sklearn.multioutput import MultiOutputRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error
import pandas as pd

# Load training data (ICES DATRAS joined to CMEMS)
df = pd.read_csv('training_data.csv')

FEATURES = ['sst_celsius', 'chlorophyll_mg_m3', 'depth_m',
            'substrate_type', 'wave_height_m', 'wind_kmh',
            'season_week', 'distance_port_km', 'historical_cpue']

TARGETS = ['plaice_kg', 'sole_kg', 'cod_kg', 'herring_kg', 'mackerel_kg']

X = df[FEATURES]
y = df[TARGETS]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

```



```

model = MultiOutputRegressor(XGBRegressor(
    n_estimators=200,
    max_depth=5,
    learning_rate=0.05,
    subsample=0.8,
    random_state=42
))

model.fit(X_train, y_train)
preds = model.predict(X_test)
print(f"MAE: {mean_absolute_error(y_test, preds):.1f} kg")

```

Why XGBoost over alternatives:

Option	Verdict	Reason
XGBoost	✅ Use this	Trains in seconds, handles missing values, outputs continuous predictions, interpretable, scikit-learn compatible
LightGBM	✅ Also fine	Slightly faster, same API
Random Forest	⚠️ Fallback	Familiar but less accurate
Neural Network	❌ Skip	Needs more data + time to set up
Gaussian Process	❌ Not in 6h	Spatially elegant but too complex for hackathon

Training Data: What Actually Exists

ICES DATRAS (Primary Source — Use This)

- Scientific bottom trawl surveys, North Sea, since 1965
- Contains: species, exact weight, lat/lon, depth, date, gear type
- **Real labeled catch data** — thousands of station records
- Free download: datras.ices.dk
- Problem: sparse (scientific stations, not commercial grounds), annual cadence

Dutch Commercial e-Logbooks (The Gold Standard — Currently Inaccessible)

- EU-mandated electronic logbooks from all licensed NL vessels >10m
- Held by **NVWA** (Dutch Food & Consumer Safety Authority)
- Not publicly available — requires formal institutional data partnership
- This is the long-term data moat to pursue post-hackathon

STECF / Eurostat Aggregated Catch Data

- Total landings per ICES rectangle per species per year
- Coarse ($0.5^\circ \times 1^\circ$ boxes) but covers commercial reality
- Free, public

Private Fisher Logbooks

- Captains' personal notebooks — hyperlocal, multi-generational, gold standard
- 100% private, guarded carefully
- Only accessible via trust + your credit incentive program over time

Honest Accuracy Ceiling (Satellite-Only Model)

Prediction Type	Accuracy
Habitat presence / absence	~70–80%
Species probability distribution	~55–65%
Yield quantity (kg)	±40–60% error

What this means for product language:

-  Say: "Zone 7B shows strong habitat conditions for plaice based on temperature, depth, and historical survey data"
-  Don't say: "Zone 7B has exactly 520kg of plaice today"

Accuracy improves to ~80–85% once logbook data accumulates (Stage 2–3).

9. ROUTE OPTIMIZATION LOGIC

Problem Framing

Given: A vessel departing from Scheveningen, targeting Plaice, with 2 fishing days and a fuel budget of €800 — find the optimal sequence of 1–3 zones that maximizes expected catch value while minimizing fuel cost, avoiding MPAs, and respecting zone capacity caps.

This is a variant of the Travelling Salesman Problem with a value-maximizing objective. For MVP, a **greedy heuristic** is used — it achieves ~85–90% of optimal in milliseconds.

Zone Value Function

SPECIES_PRICE = { # €/kg at Dutch fish auctions (IJmuiden, Urker Visafslag)

```
'plaice': 2.80,  
'sole': 8.50,  
'cod': 3.20,  
'herring': 0.90,  
'mackerel': 1.40,  
}
```

FUEL_COST_PER_KM = 1.20 # € per km for average NL beam trawler

```
def zone_net_value(zone, fisher, from_position):
```

```
    target_yield_kg = zone.yield_kg_mean * zone.p_target_species[fisher.target_species]  
    expected_revenue = target_yield_kg * SPECIES_PRICE[fisher.target_species]  
    travel_cost = haversine(from_position, zone.center) * FUEL_COST_PER_KM  
    return expected_revenue - travel_cost
```

Greedy Route Builder

```
def suggest_route(fisher, all_zones, max_zones=2):
```

```
    # Step 1: Filter candidates
```

```
    candidates = [z for z in all_zones if  
        not z.is_mpa and  
        z.slots_available > 0 and  
        z.wave_height_m < fisher.vessel_max_wave and  
        z.p_target_species.get(fisher.target_species, 0) > 0.25  
    ]
```

```
    route = []
```

```
    current_pos = fisher.home_port
```

```
    remaining_fuel_budget = fisher.fuel_budget_eur
```

```
    while len(route) < max_zones and candidates:
```

```
        # Score all candidates from current position
```

```
        scored = []
```

```
        for zone in candidates:
```

```

travel_cost = haversine(current_pos, zone.center) * FUEL_COST_PER_KM
return_cost = haversine(zone.center, fisher.home_port) * FUEL_COST_PER_KM
if (travel_cost + return_cost) > remaining_fuel_budget:
    continue # can't afford this zone + return
net_val = zone_net_value(zone, fisher, current_pos)
scored.append((net_val, zone, travel_cost))

if not scored:
    break

# Pick highest value zone
best_val, best_zone, best_cost = max(scored, key=lambda x: x[0])
route.append({
    'zone': best_zone,
    'travel_cost_eur': best_cost,
    'expected_yield_kg': best_zone.yield_kg_mean *
best_zone.p_target_species[fisher.target_species],
    'expected_revenue_eur': best_val + best_cost
})
remaining_fuel_budget -= best_cost
current_pos = best_zone.center
candidates.remove(best_zone)

# Add return leg
return_cost = haversine(current_pos, fisher.home_port) * FUEL_COST_PER_KM
route.append({'zone': 'home_port', 'travel_cost_eur': return_cost})

```

return route

Route Output UI

text

🌀 Suggested Route — Jan (Plaice)		
Departing: Scheveningen	Apr 26, 05:00	
Stop 1 — Zone 7B Score: 84		
Predicted: ~520 kg plaice		

	Travel: 87km Fuel: ~€104	
	📍 Stop 2 — Zone 9C Score: 71	
	Predicted: ~310 kg plaice	
	Travel: 43km Fuel: ~€52	
	🏠 Return to Scheveningen — 102km ~€122	

	Total fuel: ~€278	
	Est. catch: ~830 kg plaice	
	Est. revenue: ~€2,324 (@€2.80/kg)	
	Est. margin: ~71% after fuel	

	[Reserve Both Zones]	

Rendered on map as a **polyline**: Port → Zone A → Zone B → Port, with distance labels on each leg.

10. HACKATHON MVP — 6-8 HOUR BUILD PLAN

What You're Building

A web app with a scored H3 hex map of the North Sea, MPA exclusion layer, zone detail cards with species + yield, and a slot-based bidding flow. Optional: suggested route overlay.

Rule: Everything that can be mocked, is mocked. The story is real. The pipeline is explained in the pitch, not built today.

Tech Stack

Layer	Choice	Why
Framework	React + Vite	Zero config, fast
Map	Mapbox GL JS	H3 native, beautiful
H3 Grid	h3-js	Client-side hex rendering
Styling	Tailwind CSS	No custom CSS needed
State	useState / Context	No Redux needed
Data	zones.json (static) → FastAPI (if time)	No backend needed for demo
Deploy	Vercel (frontend) + Railway (API)	Live URL in 2 min

Team Split

	Person	Role	Deliverable
	Person 1	Frontend — Map + Zone Card + Route overlay	React web app on Vercel
2	Person	ML / Prediction Engine	zones.json via XGBoost on ICES + CMEMS
3	Person	Pipeline A — Environmental data	sst.csv, chlorophyll.csv, weather.csv, mpa.geojson
4	Person	Pipeline B — H3 gridding + feature assembly	hex_features.csv → zones.json

The Integration Contract (Agree in Hour 0)

json

```
{
  "hex_id": "851f9a3ffffffff",
  "lat": 53.8,
  "lng": 4.2,
  "zone_score": 84,
  "species": [
    {"name": "Plaice", "prob": 0.62},
    {"name": "Sole", "prob": 0.28},
    {"name": "Cod", "prob": 0.10}
  ],
  "yield_low_kg": 420,
  "yield_high_kg": 680,
  "slots_total": 5,
  "slots_filled": 2,
  "wave_height_m": 1.2,
  "wind_kmh": 18,
  "is_mpa": false,
  "distance_km": 87
}
```

Person 1 mocks this file immediately and builds the entire frontend against it. Persons 2/3/4 replace the mock with real data by hour 5. The frontend never changes its data contract.

Hour-by-Hour Timeline

Hour 0 — Alignment (All 4 people, 15 min)

- Agree on JSON contract above
- Person 1 generates mock zones.json (25–30 hexes, ChatGPT in 10 min)
- Everyone installs dependencies and starts

Hour 1 — Map Skeleton (Person 1)

text

- React + Vite project scaffold
- Mapbox GL centered on North Sea [4.0, 53.5], zoom 6
- Load zones.json
- Render H3 hexagons colored by zone_score (green/yellow/red)
- MPA hexes: red hatched fill

✅ Colored hex map visible in browser

Hour 2 — Zone Card (Person 1)

text

- Click hex → side panel slides in
- Shows: score, species bar, yield range, slots progress bar, wave height, wind, distance, bid button
- MPA click → "🚫 Marine Protected Area. Fines up to €50,000."

✅ Zone card populated from JSON on click

Hour 3 — Onboarding + Score Filtering (Person 1)

text

- 3-step modal on first load:
 - Step 1: Home port selection
 - Step 2: Target species (multi-select)
 - Step 3: Vessel/gear type
- Zone scores re-weighted client-side based on species preference

✅ Onboarding changes map zone prominence

Hour 4 — Bidding Flow (Person 1)

text

- "Bid for this zone" → modal:
 - Time window selector (Morning / Full Day / Overnight)
 - Slot summary + confirm button
- On confirm: slots_filled + 1, toast notification, button → "Reserved ✓"

✅ Full bidding flow works end to end

Hour 5 — Route Suggestion (Person 1, if time)

text

- "Suggest Route" button → computes top 2 scoring zones for fisher profile
- Mapbox polyline: Port → Zone A → Zone B → Port
- Route summary panel: total fuel cost, est. catch, est. revenue

- "Reserve Both Zones" button triggers 2x bidding flow

✓ Route visible on map with trip economics

Person 2 — ML Timeline

text

Hour 0–1: Download ICES DATRAS CSV + register CMEMS account

Hour 1–2: Spatial join ICES records to CMEMS SST/chlorophyll by lat/lon + month
→ training_data.csv (~thousands of labeled rows)

Hour 2–3: Train XGBoost MultiOutputRegressor
→ features: SST, chlorophyll, depth, substrate, season, wave, cpue
→ targets: plaice_kg, sole_kg, cod_kg, herring_kg, mackerel_kg

Hour 3–4: Run inference on all hex centroids from hex_features.csv
→ predictions.csv

Hour 4–5: Apply zone score formula → generate final zones.json

Hour 5–6: FastAPI wrapper (optional) or commit zones.json to repo
→ Person 1 swaps mock → real data

Person 3 — Pipeline A Timeline

text

Hour 0–2: Pull CMEMS SST + chlorophyll
→ pip install copernicusmarine
→ copernicusmarine subset for NL bbox, monthly 2022–2024
→ convert netCDF → sst_monthly.csv, chlorophyll_monthly.csv

Hour 2–3: Pull Open-Meteo Marine API (free, no key)
→ wave_height, wind_speed per NL grid point → weather_grid.csv

Hour 3–4: Pull EMODnet MPA polygons
→ WFS GeoJSON endpoint → mpa_polygons.geojson
→ Also pull bathymetry grid → depth_grid.csv

Hour 4–5: Hand all files to Person 4
→ Assist with spatial join if needed

Person 4 — Pipeline B Timeline

text

Hour 0–1: Generate H3 hex grid
→ pip install h3 geopandas shapely
→ polyfill NL North Sea bbox at resolution 5
→ ~80–120 hexes → hex_centroids.csv

- Hour 1–3: Spatial join all data sources to hex centroids
- for each hex: nearest SST, chlorophyll, depth, wave, wind
 - MPA check: point-in-polygon for each centroid
 - ICES CPUE: average historical catch rate per hex region
 - Output: hex_features.csv

Hour 3–4: Hand hex_features.csv to Person 2

- Hour 4–5: FastAPI wrapper
- GET /zones → return zones.json
 - GET /zones/{hex_id} → single zone detail
 - uvicorn main:app --reload
 - Deploy: railway up

Integration Checkpoints

Time Checkpoint

- Hour 1 JSON contract agreed. Person 1 has mock file, builds against it.
- Hour 3 Person 3+4 deliver hex_features.csv to Person 2
- Hour 5 Person 2 delivers zones.json with real scores. Person 1 swaps mock → real.
- Hour 6 Full demo flow working end-to-end with real data
- Hour 7 Deploy: Vercel (frontend) + Railway (API). Live URL confirmed.
- Hour 8 Demo rehearsal. Done.

What NOT to Build

Skip	Why
User auth / login	Unnecessary complexity
Real payment / credit system	Out of scope
Real-time AIS feed	Static dots or omit entirely
Exact kg predictions with confidence	Data doesn't support it honestly
Backend database	Static JSON is enough for demo

11. TEAM STRUCTURE & TASK DELEGATION

Python Dependencies (Persons 2, 3, 4)

bash

```
pip install xgboost scikit-learn pandas numpy h3 geopandas shapely \
    fastapi uvicorn copernicusmarine requests netCDF4
```

JS Dependencies (Person 1)

```
bash
npm create vite@latest bluevantage -- --template react
cd bluevantage
npm install mapbox-gl h3-js @deck.gl/core @deck.gl/layers tailwindcss
```

Data Download Commands (Person 3)

```
bash
# ICES DATRAS — download manually from datras.ices.dk
# Select: IBTS survey, North Sea, all years, CSV export
```

```
# CMEMS via Python client
```

```
copernicusmarine subset \
    --dataset-id cmems_mod_glo_phy_my_0.083deg_P1M-m \
    --variable thetao --variable chl \
    --minimum-longitude 2.0 --maximum-longitude 7.5 \
    --minimum-latitude 51.5 --maximum-latitude 56.0 \
    --start-datetime 2022-01-01 --end-datetime 2024-12-31 \
    --output-filename nl_northsea_env.nc
```

```
# Open-Meteo Marine (no key needed)
# GET https://marine-api.open-meteo.com/v1/marine
# ?latitude=53.5&longitude=4.5
# &hourly=wave_height,wind_speed_10m
```

12. OPEN QUESTIONS & RISKS

Question	Owner	Priority	Status
VisNed / cooperative data sharing agreements possible?	Business Dev	High	Open
ICES DATRAS granularity — per station or regional aggregate only?	Data Engineer	High	Open
VMS data access via NVWA — partnership feasibility?	Business Dev	Medium	Open
Self-reported logbook legal status vs. official e-logbook?	Legal	Medium	Open

Question	Owner	Priority	Status
Credit bidding fairness — disadvantages small operators?	Product	Medium	Open
AIS 15-min refresh — creates arbitrage window for gaming?	ML Engineer	Low	Open
North Sea shrimp (5th species) — different gear type entirely, separate product?	Product	Low	Open

13. HONEST DATA LIMITATIONS & PITCH FRAMING

What Satellite Data Can and Cannot Do

Signal	What It Captures	What It Misses
SST (CMEMS)	Surface water temperature	Temperature at fishing depth (can differ 5°C+)
Chlorophyll-a	Phytoplankton density — food chain base	Doesn't confirm which species are present
Surface currents	Water movement direction	Fish active migration behaviour
Bathymetry	Depth, substrate type	Static — no seasonal variation

Honest accuracy ceiling (satellite + ICES only):

- Habitat presence/absence: ~70–80%
- Species probability distribution: ~55–65%
- Yield quantity (kg): ±40–60% error range

The 3-Stage Path to Reliable Predictions

Stage	Data	Accuracy	Timeline
Stage 1	ICES DATRAS + CMEMS satellite	~65% directional	Launch
Stage 2	+ Fisher logbook submissions (incentive flywheel)	~75–80%	6–18 months
Stage 3	+ NVWA/RWS e-logbook partnership	~85%+	12–24 months

How to Frame This to Judges

"Our model identifies habitat suitability zones using satellite oceanographic data and ICES historical survey data — showing fishers where conditions are historically and environmentally productive for their target species. Over time, voluntary logbook submissions from our users calibrate the model to commercial fishing reality, creating a proprietary labeled dataset no competitor can replicate. The data flywheel is the moat."

This is stronger than fake precision. Judges who understand ML will respect the honesty and the flywheel narrative is a genuine defensible moat.

14. 60-SECOND DEMO SCRIPT (FOR JUDGES)

"Commercial fishing in the Netherlands loses €180M a year to inefficient trips. Fishers leave port not knowing where the fish are.

BlueVantage fixes that. [show map] This is the North Sea — every hexagon is a scored fishing zone, updated daily using sea temperature, chlorophyll levels, and historical ICES stock survey data. Green means go. Red means low yield. The hatched zones are MPAs — we hard-block them so fishers never risk a €50,000 fine.

[click a zone] Zone 7B. Score 84. Strong habitat conditions for plaice. Estimated 420–680kg per vessel-day. 2 of 5 slots taken. [click bid] I select my time window, confirm — slot reserved. The zone updates live so two fishers are never sent to the same place.

[route suggestion] Jan targets plaice out of Scheveningen. The platform suggests a 2-stop route: Zone 7B then 9C — estimated 830kg, €2,300 revenue, €278 fuel. He reserves both zones in one click.

[log catch] After the trip, Jan logs his catch. He earns priority credits for next week's bid. That data makes our predictions better for everyone — the more fishers contribute, the smarter the platform gets.

This is BlueVantage — informed fishing, fairly allocated, sustainably capped. The recommendation is just the entry point. The marketplace is the business. The logbook flywheel is the moat."

15. 5-SLIDE PITCH DECK STRUCTURE

Slide	Title	Content
1	The Problem	€180M lost annually to blind fishing in NL. 30–40% of trips underperform. Fuel wasted searching. €50k MPA fines. Screenshot of a fisher at sea with no data.
2	The Solution	Zone intelligence + self-allocation marketplace. Screenshot of the map with scored hexes + zone card. One sentence: "We tell fishers where to go — and make sure they don't all go to the same place."
3	How It Works	Simple 4-step flow diagram: Satellite + ICES data → H3 zone scoring → Fisher bids for slot → Logs catch → Model improves. Emphasize the flywheel.
4	Business Model	Freemium → €150 / €250 / €600/mo tiers. 400 addressable NL vessels. 150 vessels at Y3 = €270k ARR. LTV:CAC = 9x. Path to NVWA data partnership + B2G sustainability reports.
5	Why Now / Why Us	EU Green Deal sustainability pressure on fishing sector. NL ITQ quota system primes fishers for zone allocation thinking. ICES + CMEMS data publicly available today. Team has TU Delft oceanography + CS + data engineering coverage. First mover building the proprietary logbook dataset.

APPENDIX — QUICK REFERENCE LINKS

Resource	URL
ICES DATRAS trawl survey data	https://datras.ices.dk
Copernicus Marine (CMEMS)	https://marine.copernicus.eu
Copernicus Python client	https://pypi.org/project/copernicusmarine
Open-Meteo Marine API	https://marine-api.open-meteo.com
EMODnet Human Activities (MPA)	https://www.emodnet-humanactivities.eu
RWS Natura 2000 NL	https://www.rijkswaterstaat.nl
AISHub (free AIS feed)	https://www.aishub.net
MarineTraffic API	https://www.marinetraffic.com/en/ais-api-services
Uber H3 Python library	https://pypi.org/project/h3
Mapbox GL JS	https://docs.mapbox.com/mapbox-gl-js
XGBoost docs	https://xgboost.readthedocs.io
FastAPI docs	https://fastapi.tiangolo.com
Railway (API hosting)	https://railway.app
Vercel (frontend hosting)	https://vercel.com
STECF fisheries data	https://stecf.jrc.ec.europa.eu

END OF DOCUMENT

BlueVantage Master Project Document v1.0 — April 25, 2026
Compiled from full product research, business modeling, technical design, and hackathon planning sessions.

Next version: post-hackathon with real model accuracy metrics and pilot fisher feedback.